

— CIÊNCIA DA COMPUTAÇÃO · 2º SEMESTRE

Introdução à Programação Estruturada

Módulo 1 — Apresentando o Python

CARGA HORÁRIA
SEMANAL

4,5 h-aula

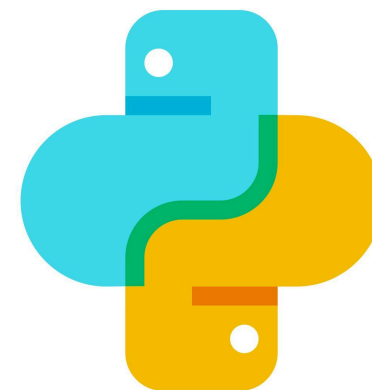
CARGA HORÁRIA
SEMESTRAL

90 h-aula

SEMESTRE

2º / CC

AULA INAUGURAL · MÓDULO 1 DE 1



PYTHON.ORG

Ementa da Disciplina

A disciplina de Introdução à Programação Estruturada apresenta aos estudantes os fundamentos da programação imperativa, utilizando a linguagem Python como ambiente de aprendizagem. O foco está na construção do raciocínio algorítmico, no domínio de estruturas de controle de fluxo e na organização modular do código. Ao longo do semestre, os alunos desenvolverão a capacidade de analisar problemas, projetar soluções estruturadas e implementá-las com clareza e correção. **A prática será conduzida via BeeCrowd (antigo URI Online Judge)**, plataforma brasileira de juiz online onde cada aluno resolverá problemas semanais em casa, com feedback automatizado.

EMENTA COMPLETA — TÓPICOS COBERTOS

 **Conceitos básicos de programação estruturada**
Algoritmos, pseudocódigo, fluxogramas.

 **Introdução ao Python**
História, características, ambiente VS Code.

 **Variáveis, tipos e operadores**
Primitivos, conversão, f-strings.

 **Estruturas de controle de fluxo**
Condicionais, laços, break/continue.

 **Funções e modularização**
Definição, parâmetros, escopo.

 **Estruturas de dados básicas**
Listas, tuplas, dicionários.

FICHA TÉCNICA

CURSO

Ciência da Computação

SÉRIE

2º semestre

DISCIPLINA

Introdução à Programação Estruturada

CARGA HORÁRIA

4,5 h-aula/semana · 90 h-aula/semestre

PLATAFORMA DE PRÁTICA

BeeCrowd (juiz online)

METODOLOGIA

Aula expositiva + demonstração ao vivo

PRÉ-REQUISITO

Lógica de Programação (1º semestre)

Carga Horária e Distribuição no Semestre

90 h-aula

CARGA TOTAL

4,5 h/semana

AULAS PRESENCIAIS

20 semanas

DURAÇÃO DO SEMESTRE

2 aulas/semana

ENCONTROS · 100 MIN CADA

DISTRIBUIÇÃO POR MÓDULO

MÓDULO	TEMA	CARGA ESTIMADA
M1 Atual	Apresentando o Python	12 h
M2	Variáveis, Tipos e Operadores	18 h
M3	Estruturas de Controle de Fluxo	22 h
M4	Funções e Modularização	20 h
M5	Estruturas de Dados Básicas	18 h

A carga horária foi distribuída para garantir que cada conceito seja praticado em aula antes de avançar. Os módulos são cumulativos — cada novo tópico se apoia nos anteriores.

Os 5 Módulos da Disciplina

Do primeiro contato com Python até o domínio de estruturas de dados fundamentais.

MÓDULO ATUAL

M1

MÓDULO 1 · 12 H

Apresentando o Python

- Conceitos básicos da linguagem
- Ambiente VS Code
- Primeiros programas

M2

MÓDULO 2 · 18 H

Variáveis, Tipos e Operadores

- Tipos primitivos (int, float, str, bool)
- Operadores aritméticos e lógicos
- Conversão de tipos

M3

MÓDULO 3 · 22 H

Estruturas de Controle de Fluxo

- Condicionais (if/elif/else)
- Laços de repetição (while, for)
- Break e continue

M4

MÓDULO 4 · 20 H

Funções e Modularização

- Definição e chamada
- Parâmetros e retorno
- Escopo de variáveis

M5

MÓDULO 5 · 18 H

Estruturas de Dados Básicas

- Listas
- Tuplas
- Dicionários

Módulo 1 — Objetivos de Aprendizagem

Ao final desta aula inaugural, espera-se que cada estudante seja capaz de compreender o que é a linguagem Python, reconhecer suas principais vantagens em relação a outras linguagens e entender como o ambiente de desenvolvimento será configurado. Esta é uma aula **EXPOSITIVA com demonstração ao vivo pelo professor** — não há ida ao laboratório e os alunos não precisam trazer computador para a sala. Toda a instalação, configuração e primeiros programas serão demonstrados no PC da apresentação em tempo real. Os alunos deverão reproduzir o passo a passo posteriormente em casa, utilizando o material de apoio disponibilizado.

i **IMPORTANTE:** Aula 100% expositiva com demonstração ao vivo. A prática acontece em casa, via plataforma BeeCrowd — apresentada no final desta aula.

01



Compreender o que é Python

Entender a história, a filosofia e as principais características técnicas da linguagem. Reconhecer o papel do Python no cenário contemporâneo de desenvolvimento de software.

02



Identificar as vantagens do Python

Comparar Python com linguagens como C e Java em termos de legibilidade, produtividade e ecossistema. Perceber por que Python é a escolha didática para iniciantes.

03



Conhecer o VS Code

Entender o que é um Ambiente de Desenvolvimento Integrado (IDE) e por que o Visual Studio Code será nossa ferramenta oficial. Reconhecer suas principais funcionalidades.

04



Acompanhar a instalação demonstrada

Acompanhar passo a passo a instalação do Python e do VS Code demonstrada ao vivo pelo professor. O material completo ficará disponível para reprodução em casa.

05



Conhecer o BeeCrowd

Entender o que é a plataforma BeeCrowd, como criar conta e como submeter soluções. Visualizar os problemas recomendados para o Módulo 1 e a mecânica de avaliação.

CAPÍTULO

01

DE 05

História, filosofia, características técnicas e ecossistema de uma das linguagens mais populares do mundo.

NESTE CAPÍTULO

 Definição e história

 O Zen do Python

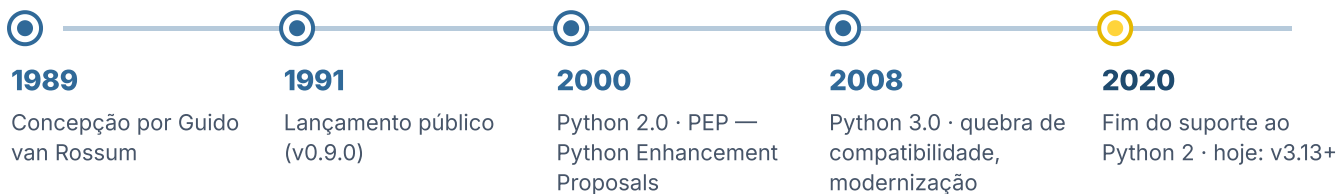
 Características técnicas

 Áreas de aplicação

O que é Python?

Python é uma linguagem de programação de alto nível, interpretada, multiparadigma e de tipagem dinâmica, criada com foco em legibilidade e simplicidade. Foi concebida no final de 1989 e lançada oficialmente em fevereiro de 1991 por Guido van Rossum, então pesquisador no Centrum Wiskunde & Informatica (CWI), nos Países Baixos. O nome da linguagem não vem da cobra, mas sim do grupo de comédia britânico Monty Python, do qual Guido era fã assumido. Desde então, Python tornou-se uma das linguagens mais utilizadas no mundo, ocupando consistentemente as primeiras posições em índices como TIOBE e Stack Overflow Developer Survey.

LINHA DO TEMPO



Guido van Rossum — criador do Python e BDFL (Benevolent Dictator for Life) até 2018.

O Zen do Python

” *Beautiful is better than ugly. Explicit is better than implicit. Simple is better than complex. Complex is better than complicated.* ”

— Tim Peters, PEP 20 (The Zen of Python)

O Zen do Python é um conjunto de 19 aforismos escritos por Tim Peters em 1999 que sintetizam a filosofia de design da linguagem. Para visualizá-lo, basta executar o comando `import this` em qualquer interpretador Python. Esses princípios guiam não apenas o desenvolvimento da própria linguagem, mas também o estilo de código que toda comunidade Python valoriza.

```
>>> import this
```

6 PRINCÍPIOS-CHAVE



Legibilidade conta

Código é lido mais vezes do que escrito.



Explícito > Implícito

Deixe claro o que o código faz.



Simplicidade primeiro

Reduza complexidade sempre que possível.



Praticidade vence pureza

Útil importa mais que ortodoxo.



Erros não devem passar silenciosos

Exceto se explicitamente silenciados.



Casos especiais não quebram regras

Embora a praticidade possa vencer a pureza.

Características Técnicas Fundamentais

Para entender por que Python se comporta como se comporta — e por que é tão adequado para iniciantes — é preciso conhecer suas principais características técnicas. Essas propriedades definem como o código é executado, como a memória é gerenciada e que paradigmas a linguagem suporta. A combinação dessas decisões de design é o que torna Python produtivo, flexível e acessível.



01

Interpretada

O código-fonte é executado diretamente por um interpretador, sem etapa de compilação prévia para código de máquina. Isso torna o ciclo de desenvolvimento mais rápido — escreva, execute, veja o resultado. O interpretador padrão é o **CPython**, escrito em C.



02

Tipagem dinâmica

Variáveis não precisam ter seu tipo declarado explicitamente; o tipo é inferido no momento da atribuição. Uma mesma variável pode mudar de tipo ao longo da execução. Isso reduz a verbosidade, mas exige atenção do programador.



03

Multiparadigma

Suporta programação imperativa, orientada a objetos e funcional. O desenvolvedor pode escolher o estilo mais adequado ao problema. Não há imposição de um paradigma único.



04

Multiplataforma

Roda em Windows, macOS, Linux, BSD e até em microcontroladores (**MicroPython**). O mesmo código-fonte pode ser executado em qualquer plataforma sem modificação. Distribuições oficiais estão disponíveis em python.org.



05

Gerenciamento automático de memória

Um coletor de lixo (*garbage collector*) libera automaticamente a memória de objetos que não estão mais em uso. Não é necessário fazer **malloc/free** manualmente como em C. Isso reduz vazamentos de memória e erros comuns.



06

Tipagem forte

Operações entre tipos incompatíveis geram erro (ex.: `'a' + 1` falha). Isso evita bugs sutis que ocorrem em linguagens com tipagem fraca como JavaScript. O programador precisa converter explicitamente.

Áreas de Aplicação

Python é uma linguagem generalista — sua flexibilidade permite aplicá-la em domínios muito distintos, do servidor web ao satélite em órbita. Conhecer essas áreas ajuda a entender por que dominar Python abre portas para múltiplas carreiras em tecnologia.



Ciência de Dados

Análise, limpeza, visualização e modelagem estatística de grandes volumes de dados. Python é a linguagem dominante nesse campo, presente em 90% dos fluxos de trabalho de cientistas de dados segundo o KDnuggets Survey. Bibliotecas como NumPy e Pandas tornaram-se padrão de mercado.

BIBLIOTECAS

NumPy · Pandas · Matplotlib · Seaborn · Plotly



Inteligência Artificial & ML

Treinamento de modelos de aprendizado de máquina e deep learning, processamento de linguagem natural, visão computacional e sistemas de recomendação. Os principais frameworks do mercado são Python-first. Pesquisadores e engenheiros de IA usam Python diariamente.

BIBLIOTECAS

TensorFlow · PyTorch · scikit-learn · Keras · Hugging Face



Desenvolvimento Web

Construção de backends robustos, APIs REST e GraphQL, e aplicações web full-stack. Frameworks maduros oferecem roteamento, ORM, autenticação e segurança prontos para produção. Vários gigantes da web rodam Python no servidor.

FRAMEWORKS

Django · Flask · FastAPI · Tornado · Bottle



Automação & Scripts

Automação de tarefas repetitivas, scraping de dados da web, processamento de arquivos, integrações entre sistemas. Python substitui com vantagem scripts em shell, batch e VBScript. É a linguagem mais usada por sysadmins e DevOps.

FERRAMENTAS

Selenium · BeautifulSoup · PyAutoGUI · Ansible



Jogos & Entretenimento

Desenvolvimento de jogos 2D, prototipagem rápida e lógica de jogos indie. Embora não seja a escolha para AAA (que usam C++/C#), Python é excelente para game jams e aprendizado. Também usado em pipelines de animação na Pixar e ILM.

BIBLIOTECAS

Pygame · Arcade · Panda3D · Cocos2d



Aplicações científicas e espaciais

Simulações físicas, bioinformática, astronomia, robótica e sistemas embarcados. A NASA usa Python em missões espaciais; o CERN o usa no LHC para análise de colisões. É a linguagem padrão em muitos laboratórios de pesquisa.

BIBLIOTECAS

SciPy · Astropy · Biopython · MicroPython

Quem Usa Python no Mundo Real

Python não é apenas uma linguagem acadêmica — é uma peça central da infraestrutura das maiores empresas de tecnologia do planeta. Conhecer esses casos reais ajuda a dimensionar o impacto da linguagem no mercado de trabalho. Das startups às Big Techs, Python aparece em algum lugar crítico do stack.

GOOGLE

BIG TECH

USO

Infraestrutura e ferramentas internas

Python é uma das três linguagens oficiais do Google (junto com C++ e Java). Muitas ferramentas internas de build, teste e automação são escritas em Python.

NETFLIX

STREAMING

USO

Recomendação e orquestração

O algoritmo de recomendação, análise de visualização e orquestração de deploy são feitos em Python. A Netflix mantém diversos projetos open-source em Python.

INSTAGRAM

SOCIAL

USO

Backend web (Django)

O backend do Instagram é um dos maiores deployments de Django do mundo, servindo bilhões de requisições por dia. Também usam Python para análise e segurança.

SPOTIFY

ÁUDIO

USO

Serviços de rádio e recomendação

Cerca de 80% dos serviços backend do Spotify usam Python, principalmente para recomendação musical e análise de preferências.

NASA

CIÊNCIA

USO

Missões espaciais e análise científica

Python é usado em missões como o Telescópio James Webb e o rover Curiosity, para análise de imagens e telemetria.

NUBANK

BRASIL

USO

Backend bancário e ciência de dados

Maior banco digital da América Latina, o Nubank mantém grande parte de sua stack em Python (Django + Clojure) e é referência em Python no Brasil.

CAPÍTULO

02

DE 05

Vantagens do Python

Por que Python se tornou a linguagem didática mais adotada no mundo — e por que ela vale a pena para você, aluno de CC.



Legibilidade e sintaxe



Comparativo com C e Java



Ecosistema e bibliotecas



Comunidade e mercado

Legibilidade e Sintaxe Limpa

Uma das marcas registradas do Python é a legibilidade — o código se parece muito com pseudocódigo, permitindo que o leitor entenda a lógica sem precisar decifrar símbolos. Isso reduz a carga cognitiva e torna a manutenção mais barata. Para iniciantes, significa menos tempo lutando contra a sintaxe e mais tempo aprendendo a lógica.

<> PYTHON

5 linhas úteis

```
nome = input('Seu nome: ')
idade = int(input('Sua idade: '))

if idade ≥ 18:
    print(f'{nome} é maior de idade.')
else:
    print(f'{nome} é menor de idade.')
```

C

14 linhas (com cerimônia)

```
#include <stdio.h>
#include <string.h>

int main() {
    char nome[50];
    int idade;
    printf('Seu nome: ');
    scanf('%s', nome);
    printf('Sua idade: ');
    scanf('%d', &idade);
    if (idade ≥ 18)
        printf('%s é maior de idade.\n', nome);
    else
        printf('%s é menor de idade.\n', nome);
    return 0;
}
```

 **Sem** ponto-e-vírgula obrigatório **Sem** chaves para blocos (usa indentação) **Sem** declaração explícita de tipo

Python vs C vs Java

Para situar Python no ecossistema das linguagens, comparamos suas características com duas outras linguagens muito ensinadas em cursos de CC: C (clássica, de baixo nível) e Java (orientada a objetos, enterprise). A tabela destaca as diferenças mais relevantes para iniciantes.

Critério	★ Python	C	Java
🔧 Paradigma principal	Multiparadigma (imperativa, OO, funcional)	Imperativa estruturada	Orientada a objetos
🏗️ Tipagem	Dinâmica e forte	Estática e fraca	Estática e forte
🏃 Execução	Interpretada (mais lenta)	Compilada (mais rápida)	Compilada para bytecode + JVM
📈 Curva de aprendizado	Suave (ideal para iniciantes)	Íngreme (ponteiros, memória manual)	Moderada (verbosa, mas consistente)
💻 Linhas de código (mesmo programa)	~5 linhas	~14 linhas	~10 linhas
🧩 Gerenciamento de memória	Automático (GC)	Manual (malloc/free)	Automático (GC)
🌐 Comunidade ativa	Enorme e crescente	Estável e madura	Muito grande, em transição
👛 Mercado (Brasil)	Alta demanda (dados, IA, web)	Demanda estável (sistemas, embarcados)	Alta demanda (enterprise, Android)

Nenhuma linguagem é “melhor” que a outra em absoluto — cada uma foi projetada para resolver problemas diferentes. Python é a melhor porta de entrada para aprender lógica de programação; C e Java aparecem naturalmente em disciplinas posteriores do curso.

Bibliotecas e o PyPI

Uma das maiores forças do Python é seu ecossistema de bibliotecas de terceiros, distribuídas via PyPI (Python Package Index). Qualquer funcionalidade que você precise — de machine learning a processamento de imagens — provavelmente já tem uma biblioteca pronta. O comando `pip install` traz o pacote para o seu projeto em segundos.

500.000+

Pacotes disponíveis no PyPI

100 bi/ano

Downloads de pacotes Python no mundo

pip install

Comando único para instalar qualquer pacote

 WEB	 DADOS & CIÊNCIA	 IA & MACHINE LEARNING	 AUTOMAÇÃO & DEVOPS
Django Framework full-stack completo	NumPy Computação numérica	TensorFlow Deep learning (Google)	Selenium Automação de browser
Flask Microframework minimalista	Pandas Análise tabular	PyTorch Deep learning (Meta)	BeautifulSoup Parsing de HTML
FastAPI APIs modernas e tipadas	Matplotlib Visualização 2D	scikit-learn ML clássico	Ansible Provisionamento
Requests Cliente HTTP simples	Jupyter Notebooks interativos	Transformers NLP (Hugging Face)	Celery Filas assíncronas

Para um iniciante, basta saber que existe um universo de bibliotecas disponível. Conforme avançamos no curso, faremos nosso primeiro `pip install` e exploraremos algumas dessas bibliotecas.

Por que aprender Python é um bom investimento

Python possui uma das maiores e mais ativas comunidades de programação do mundo. Isso se traduz em tutoriais abundantes, fóruns movimentados (Stack Overflow, Reddit, Discord), eventos presenciais (PyCon Brasil, Python Brasil) e oportunidades de networking. Para iniciantes, significa que quase qualquer dúvida já foi respondida em algum lugar da internet.

📈 MERCADO DE TRABALHO — BRASIL 2025

#1

Linguagem mais demandada no LinkedIn (BR)

+47%

Crescimento de vagas vs 2023

R\$ 8–12k

Faixa salarial sênior (dados/IA)

18 mil+

Vagas ativas (LinkedIn BR)

Para além do emprego formal, Python abre portas para freelas, projetos open-source, pesquisa acadêmica e empreendedorismo. Muitos alunos do 2º semestre já conseguem estágios remunerados usando Python como diferencial.

🚀 COMO COMEÇAR A CONSTRUIR PORTFÓLIO

- ✓ Crie conta no GitHub e publique seus primeiros programas
- ✓ Participe do Python Brasil (Discord gratuito)
- ✓ Assine a newsletter Python Weekly



Laboratório de CC — onde a prática acontece.

CAPÍTULO

03

DE 05

Ambiente de Desenvolvimento

O Visual Studio Code — nossa ferramenta oficial. Por que escolhemos uma IDE e o que ela faz por você durante o aprendizado.

 O que é o VS Code

 Recursos essenciais

 Extensões para Python

Visual Studio Code (VS Code)

O Visual Studio Code é um editor de código-fonte gratuito, open-source e multiplataforma, desenvolvido pela Microsoft. Embora seja tecnicamente um editor (não uma IDE completa), suas extensões o transformam em uma IDE poderosa para qualquer linguagem. É o editor mais usado no mundo segundo o Stack Overflow Developer Survey (4 anos consecutivos). Para o iniciante, oferece o equilíbrio ideal entre simplicidade e recursos.

POR QUE VS CODE E NÃO OUTRO?



Gratuito e open-source

Sem custo de licença, mantido pela Microsoft e pela comunidade.



Leve e rápido

Inicializa em segundos, consome pouca memória. Funciona bem mesmo em máquinas modestas.



Extensível

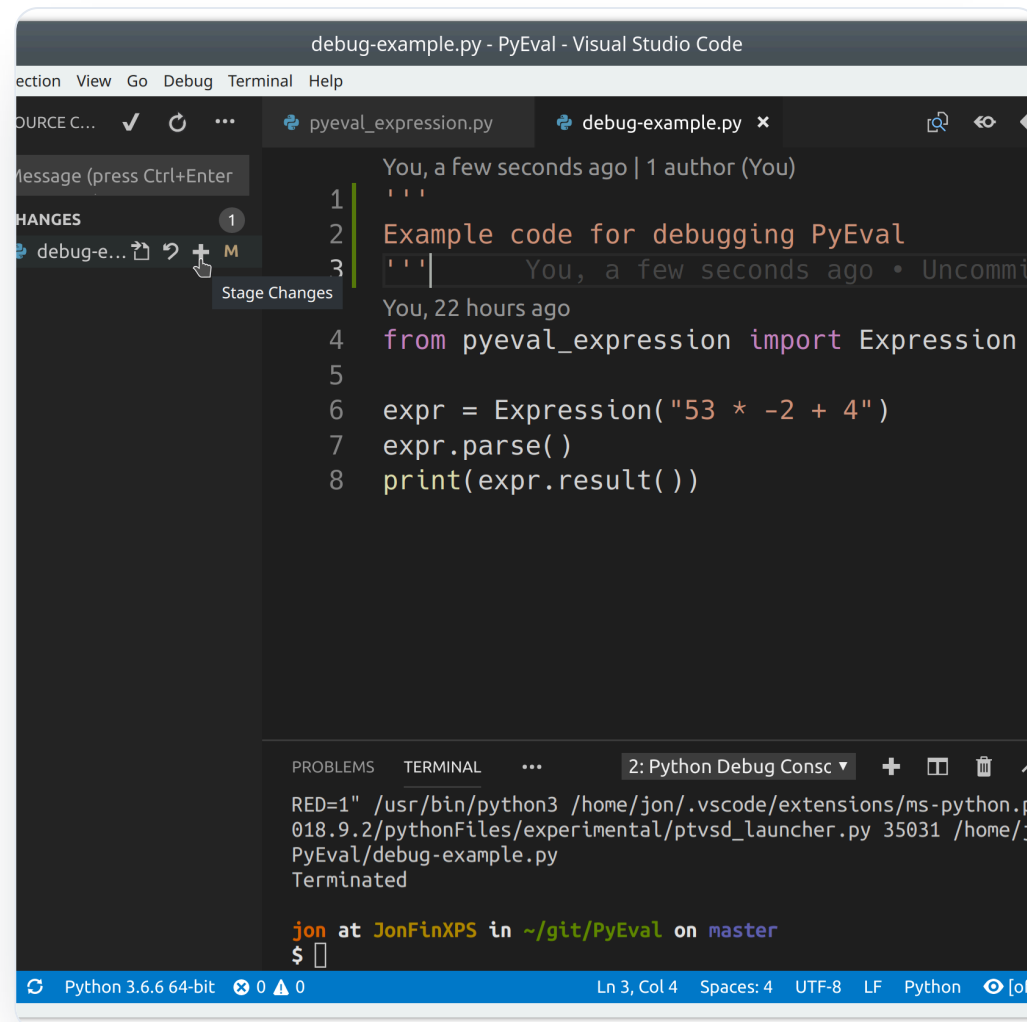
Mais de 40.000 extensões no Marketplace. Vira editor para qualquer linguagem.



Padrão de mercado

Usado por empresas como Microsoft, Google, Facebook e na maioria das universidades.

Outras opções existem — PyCharm (JetBrains), Spyder, IDLE (que vem com Python) — mas o VS Code é o que você encontrará no mercado de trabalho. Padronizar a turma em uma única ferramenta facilita o suporte do professor e a colaboração entre alunos.



VS Code com código Python — destaque de sintaxe, IntelliSense e terminal integrado.

Funcionalidades Essenciais do VS Code

Para aproveitar o VS Code ao máximo desde a primeira aula, é importante conhecer os recursos que ele oferece nativamente. Essas funcionalidades reduzem erros, aceleram a escrita e facilitam o aprendizado.



01

Realce de sintaxe

Coloriza palavras-chave, strings, números e comentários de forma distinta. Isso torna o código muito mais legível e ajuda a identificar erros visuais rapidamente. Cada linguagem tem seu próprio esquema de cores.



02

IntelliSense (autocompletar)

Sugere nomes de variáveis, funções e métodos enquanto você digita. Reduz erros de digitação e acelera a escrita. Para iniciantes, funciona como um “lembrete” do que existe na linguagem.



03

Terminal integrado

Permite executar comandos e rodar seus programas Python sem sair do editor. Combina edição e execução no mesmo ambiente. Aumenta significativamente a produtividade.



04

Depurador (debugger)

Executa o programa passo a passo, inspecionando variáveis em tempo real. Essencial para entender o fluxo de execução e encontrar bugs. Suporta breakpoints, watch e call stack.



05

Formatação automática

Reorganiza indentação, espaçamento e quebras de linha automaticamente ao salvar. Para Python, segue o padrão PEP 8 (que veremos adiante). Mantém o código consistente.



06

Controle de versão (Git)

Integração nativa com Git e GitHub. Permite commit, pull, push e resolução de conflitos pela interface. Facilita o versionamento desde o primeiro dia.

Extensões Essenciais para Python

Extensões são pacotes que adicionam funcionalidades específicas ao VS Code. Para nossa disciplina, vamos instalar um conjunto mínimo de extensões que transformam o VS Code em uma IDE Python completa. Todas são gratuitas e disponíveis no Marketplace.

	Python por Microsoft	A extensão oficial do Python para VS Code. Adiciona IntelliSense, debugging, linting, formatação, navegação por código e execução de células Jupyter. É o ponto de partida obrigatório — sem ela, o VS Code não reconhece Python.	! OBRIGATÓRIA
	Pylance por Microsoft	Servidor de linguagem Python de alta performance, baseado em Type Checking. Oferece autocompletar inteligente, verificação de tipos em tempo real e documentação inline. Substitui o antigo Jedi com ganhos significativos de velocidade.	! OBRIGATÓRIA
	autoDocstring por Nils Werner	Gera automaticamente o esqueleto de docstrings (documentação de funções) em Python. Acelera a escrita de código documentado. Suporta padrões Google, NumPy e Sphinx.	★ RECOMENDADA
	Code Runner por Jun Han	Permite executar o arquivo Python atual com um único clique (botão play no canto superior direito). Útil para testar programas pequenos rapidamente. Não substitui o terminal para programas com entrada de dados.	★ RECOMENDADA
	Material Icon Theme por Philipp Kief	Aplica ícones coloridos e identificáveis para arquivos e pastas no explorador. Diferencia visualmente arquivos .py, .txt, .json, etc. Puramente estético, mas melhora muito a navegação.	✏ OPCIONAL

Veremos como instalar cada uma dessas extensões nos próximos slides, junto com o passo a passo da configuração do ambiente.

CAPÍTULO

04

DE 06

Instalação Passo a Passo

Demonstração ao vivo pelo PC da apresentação. Vocês devem assistir, anotar dúvidas e reproduzir o passo a passo em casa — não há laboratório nesta aula.



Instalar Python



Instalar VS Code



Configurar extensão



Verificar instalação



NÃO HÁ IDA AO LABORATÓRIO. A instalação será demonstrada pelo professor no PC da apresentação. Os alunos reproduzem em casa usando o material de apoio.

Instalando o Python 3

O interpretador Python é o programa que executa seus scripts. Sem ele instalado, nada funciona. Vamos demonstrar a instalação em Windows, macOS e Linux — acompanhem a lógica; o passo a passo completo ficará disponível para reprodução em casa.

Windows 10 / 11

5 PASSOS

- 1 Acesse `python.org/downloads`**
O site detecta Windows e oferece o instalador automaticamente. Baixe a versão mais recente (3.13+ no momento desta aula).
- 2 Execute o instalador (.exe)**
Antes de clicar em “Install Now”, **marque** a caixa `Add Python to PATH` — isso é essencial.
- 3 Aguarde a instalação**
Cerca de 1–2 minutos. Ao final, clique em “Disable path length limit” se aparecer.
- 4 Verifique no PowerShell**
Abra o PowerShell e digite `python --version`
`Python 3.13.x`
- 5 Teste o interpretador**
Digite `python` para entrar no REPL (`>>>`). Teste: `print('Olá, Python!')`.

macOS & Linux

5 PASSOS

- 1 macOS: use Homebrew**
Se ainda não tem Homebrew: instale via `brew.sh`. Depois: `brew install python`
- 2 Linux (Ubuntu/Debian)**
Abra o terminal e digite: `sudo apt update && sudo apt install python3 python3-pip`
- 3 Linux (Fedora)**
No terminal: `sudo dnf install python3 python3-pip`
- 4 Verifique a instalação**
No terminal: `python3 --version`
`Python 3.x.y`
- 5 Teste o REPL**
Digite `python3` · Teste: `print('Olá, Python!')` · Use `Ctrl+D` ou `exit()` para sair.

 **ATENÇÃO — DEMONSTRAÇÃO AO VIVO:** O professor está executando este passo agora no PC da apresentação. Acompanhem apenas visualmente; a reprodução na máquina de vocês será feita em casa.

No Linux/macOS o comando é `python3`, não `python` (que pode apontar para Python 2 legado em alguns sistemas).

Instalando o Visual Studio Code

Com o interpretador Python funcionando, o próximo passo é instalar o editor VS Code. O processo é simples e idêntico em todos os sistemas operacionais. A instalação será demonstrada agora ao vivo — vocês repetem em casa.

01



Baixe o instalador

Acesse code.visualstudio.com. O site detecta automaticamente seu sistema operacional e oferece o download correto. Clique no botão azul grande de download.

02



Execute o instalador

Windows: .exe — aceite a licença, escolha o caminho padrão. macOS: .zip — arraste VS Code para a pasta Applications. Linux: .deb ou .rpm — instale via gerenciador de pacotes.

03



Marque opções adicionais (Windows)

Na tela "Select Additional Tasks", marque "Add to PATH" (essencial), "Create desktop icon" (opcional), "Register Code as editor for supported file types" (recomendado).

04



Inicie o VS Code

Abra o VS Code a partir do menu Iniciar / Launchpad / Activities. Na primeira execução, ele pedirá para escolher um tema (claro ou escuro) e importar configurações — pule por enquanto.

05



Verifique a instalação

No VS Code, abra o terminal integrado (Ctrl+` ou Cmd+`) e digite: `code --version`. Deve mostrar três linhas com a versão do VS Code, commit hash e arquitetura x64/arm64.



DICA — DEMONSTRAÇÃO AO VIVO: O professor está baixando e instalando o VS Code agora. Prestem atenção às opções marcadas no passo 03 — esquecer "Add to PATH" é o erro mais comum. Para abrir VS Code via terminal: `code .`

Configurando a Extensão Python

Com o VS Code instalado, precisamos adicionar suporte a Python. Isso é feito instalando a extensão oficial e configurando o interpretador correto.

Demonstração ao vivo agora — vocês reproduzem em casa.

01



Abra o painel de extensões

No VS Code, clique no ícone de extensões na barra lateral esquerda (ou `Ctrl+Shift+X`). Você verá o Marketplace com milhares de extensões.

02



Pesquise por "Python"

Na barra de pesquisa, digite "Python". A primeira opção será "Python" da Microsoft, com mais de 100 milhões de downloads. Clique nela.

03



Instale a extensão

Clique no botão verde "Install". A instalação leva poucos segundos. A extensão PyLance será instalada automaticamente como dependência.

04



Selecione o interpretador

Após instalar, abra a paleta de comandos (`Ctrl+Shift+P` / `Cmd+Shift+P`) e digite "Python: Select Interpreter". Escolha a versão do Python que você instalou (ex.: Python 3.13.x).

05



Teste com um arquivo .py

Crie um arquivo novo (`Ctrl+N`), salve como `teste.py` (`Ctrl+S`). Escreva `print('VS Code + Python funcionando!')` e execute com o botão play no canto superior direito. Deve aparecer no terminal integrado.



DEMONSTRAÇÃO AO VIVO: O professor está instalando a extensão agora. Notem como o PyLance é instalado automaticamente junto — isso é esperado. O VS Code só reconhece Python se o arquivo tiver a extensão `.py`.

Verificando a Instalação

Com tudo instalado, vamos validar que o ambiente está 100% funcional executando o clássico **"Hello, World!"** ao vivo no PC do professor. Existem três formas de rodar código Python — todas serão demonstradas agora. Em casa, vocês devem reproduzir e validar que as três funcionam.

REPL — Read-Eval-Print Loop

MODO INTERATIVO

```
$ python
>>> print('Olá, Python!')
Olá, Python!
>>> 2 + 3
5
>>> exit()
```

Abrir o interpretador Python diretamente no terminal digitando python (ou python3 no Mac/Linux). Cada linha digitada é executada imediatamente, mostrando o resultado. Ideal para testar comandos curtos e explorar a linguagem.

Arquivo executável

.PY

```
# arquivo: ola.py
print('Olá, Python!')
print('Meu primeiro programa.')
```

```
$ python ola.py
Olá, Python!
Meu primeiro programa.
```

Escrever o código em um arquivo .py e executá-lo com o comando python arquivo.py. É a forma padrão de rodar programas reais. Permite reutilizar e compartilhar o código.

Direto do VS Code

IDE

[arquivo ola.py aberto no VS Code]

- clique no botão ▶ Play no canto superior direito
- ou pressione Ctrl+F5

[Terminal integrado do VS Code]

```
$ /usr/bin/python3 ola.py
Olá, Python!
Meu primeiro programa.
```

Com a extensão Python instalada, basta abrir o arquivo .py no VS Code e clicar no botão play. O VS Code roda o arquivo no terminal integrado. É a forma mais produtiva durante o desenvolvimento.

 **DEMONSTRAÇÃO AO VIVO:** O professor está executando os três métodos agora. Acompanhem no projetor. Em casa, vocês devem repetir cada um dos três e validar que funciona.

— CAPÍTULO

05

de 06

Primeiros Programas

Quatro programas clássicos demonstrados ao vivo no VS Code — vocês assistem, anotam e reproduzem em casa.

Hello World + comentários

📦 Variáveis e operadores

📄 Entrada e saída

🔗 Estruturas condicionais

📺 CADA EXEMPLO SERÁ DIGITADO E EXECUTADO AO VIVO PELO PROFESSOR. Acompanhem no projetor; o código-fonte completo de cada programa ficará disponível no material de apoio.

Hello World + Comentários

📄 arquivo: hello.py

```
# Este é um comentário de linha única
# Comentários são ignorados pelo interpretador

print('Olá, Mundo!') # comentário ao lado do código

"""Este é um comentário
de múltiplas linhas.
Pode ser usado como docstring."""

print('Bem-vindo ao Python!')
```

📄 print() — a função de saída

`print()` é uma função embutida (built-in) que exibe texto na tela. Recebe como argumento uma string (texto entre aspas) e a imprime no console. Pode receber múltiplos argumentos separados por vírgula.

📄 Comentários

Linhas iniciadas com `#` são comentários — ignoradas pelo interpretador, servem para documentar o código para humanos. Strings triplas (`"""`) são usadas como docstrings para documentar funções e módulos. Boa prática: comente o **porquê**, não o **quê**.

📄 Strings em Python

Strings podem ser delimitadas por aspas simples (`'texto'`) ou duplas (`"texto"`). Para strings que contêm aspas, use o tipo oposto ou escape com barra invertida (`\`). Ex.: `"Ele disse 'olá'"` ou `'Ele disse "olá"'` .

Variáveis e Operadores Aritméticos

arquivo: variaveis.py

```
# Atribuição de variáveis
nome = 'Maria'
idade = 20
altura = 1.68
estudante = True

# Operadores aritméticos
soma = 10 + 5
subtracao = 10 - 3
multiplicacao = 4 * 6
divisao = 10 / 3          # divisão real (float)
divisao_inteira = 10 // 3 # apenas a parte inteira
resto = 10 % 3            # resto da divisão
potencia = 2 ** 8         # 2 elevado a 8

# Saída formatada (f-string)
print(f'{nome} tem {idade} anos.')
print(f'Altura: {altura} m')
print(f'10 / 3 = {divisao:.2f}')
```

Variáveis em Python

Variáveis em Python são criadas no momento da atribuição (=). Não é necessário declarar o tipo — o interpretador infere automaticamente. O tipo de uma variável pode mudar ao longo do programa (tipagem dinâmica).

Tipos primitivos básicos

str	Texto (ex.: 'Maria')
int	Número inteiro (ex.: 20)
float	Número decimal (ex.: 1.68)
bool	Verdadeiro/Falso (True / False)

Operadores aritméticos

+	Adição
-	Subtração
*	Multiplicação
/	Divisão (sempre retorna float)
//	Divisão inteira (descarta decimais)
%	Módulo (resto da divisão)
**	Potenciação

f-strings

f-strings (formatted strings) permitem incorporar variáveis diretamente em texto, prefixando a string com `f` e usando `{variavel}`. Suportam formatação: `{divisao:.2f}` exibe 2 casas decimais. Disponível desde Python 3.6.

Entrada e Saída de Dados

arquivo: entrada_saida.py

```
# Entrada de dados com input()
nome = input('Digite seu nome: ')
idade = input('Digite sua idade: ')

# input() SEMPRE retorna string
# Precisamos converter para int
idade = int(idade)

# Cálculo
ano_nascimento = 2025 - idade

# Saída com f-string
print(f'Olá, {nome}!')
print(f'Você tem {idade} anos.')
print(f'Provável ano de nascimento: {ano_nascimento}')

# Calculadora simples
n1 = float(input('Primeiro número: '))
n2 = float(input('Segundo número: '))
print(f'Soma: {n1 + n2}')
print(f'Produto: {n1 * n2}')
```

input() — função de entrada

`input()` é a função embutida que lê dados do teclado. Recebe como argumento opcional um texto de prompt (pergunta) exibido ao usuário. **SEMPRE retorna uma string**, mesmo que o usuário digite números.

↔ Conversão de tipos

<code>int(x)</code>	Converte x para inteiro (ex.: <code>int('20')</code> → 20)
<code>float(x)</code>	Converte x para decimal (ex.: <code>float('3.14')</code> → 3.14)
<code>str(x)</code>	Converte x para string (ex.: <code>str(42)</code> → '42')
<code>bool(x)</code>	Converte para booleano (ex.: <code>bool(0)</code> → False)

Por que converter?

Como `input()` retorna string, se o usuário digitar `5 + 3` o resultado seria `53` (concatenação), não `8` (soma). Por isso é essencial converter para `int` ou `float` quando se espera número. Esquecer essa conversão é o **bug número 1 de iniciantes**.

” Saída com `print()` e f-strings

`print()` exibe no console. Com f-strings, podemos incorporar variáveis e expressões diretamente: `f'Soma: {n1 + n2}'` calcula e exibe o resultado. É a forma idiomática moderna de formatar saída em Python.

Estruturas Condicionais (if / elif / else)

arquivo: condicional.py

```
# Entrada
nota = float(input('Digite sua nota (0-10): '))

# Estrutura condicional completa
if nota >= 9.0:
    conceito = 'A'
    situacao = 'Aprovado com louvor'
elif nota >= 7.0:
    conceito = 'B'
    situacao = 'Aprovado'
elif nota >= 5.0:
    conceito = 'C'
    situacao = 'Recuperação'
else:
    conceito = 'D'
    situacao = 'Reprovado'

# Saída
print(f'Conceito: {conceito}')
print(f'Situação: {situacao}')

# Operadores lógicos
frequencia = float(input('Frequência (%): '))
if nota >= 7.0 and frequencia >= 75:
    print('Aprovado por nota e frequência.')
```

Sintaxe do if

Python usa `if`, `elif` (else if) e `else` para ramificação condicional. A condição após `if` deve resultar em um valor booleano (`True` / `False`). A indentação (4 espaços) define o bloco de código pertencente a cada condição — não há chaves como em C/Java.

Comparação

<code>=</code>	Igual a
<code><math>\neq</math></code>	Diferente de
<code>></code>	Maior que
<code><</code>	Menor que
<code>>=</code>	Maior ou igual
<code><=</code>	Menor ou igual

Lógicos

<code>and</code>	E lógico (ambas verdadeiras)
<code>or</code>	OU lógico (pelo menos uma)
<code>not</code>	NÃO lógico (inverte)

Indentação é obrigatória

Em Python, a indentação não é apenas estética — é parte da sintaxe. Linhas com mesma indentação pertencem ao mesmo bloco. Use sempre **4 espaços** (não tabs) por nível — é o padrão PEP 8.

CAPÍTULO

06

DE 06

A plataforma oficial de prática da disciplina. Onde vocês resolverão problemas de programação toda semana, com correção automatizada e ranking individual.

NESTE CAPÍTULO

-  O que é o BeeCrowd
-  Como usar (passo a passo)
-  Como submeter soluções
-  Problemas recomendados (Módulo 1)

AVISO IMPORTANTE

 **prática da disciplina acontece no BeeCrowd** Cada aluno deve criar conta até a próxima aula e resolver os 6 problemas introduzidos do Módulo 1.

O que é o BeeCrowd

BeeCrowd (antigo URI Online Judge) é uma plataforma brasileira de juiz online gratuita, criada em **2007** e utilizada por milhões de estudantes de programação em todo o mundo. O sistema oferece mais de **2.500 problemas** de programação categorizados por dificuldade (iniciante, intermediário, avançado) e por tópico (estruturas, strings, grafos, etc.). Cada problema vem com um enunciado detalhado, exemplos de entrada e saída esperada, e limite de tempo/memória. Ao submeter uma solução, o sistema a compila e testa automaticamente contra dezenas de casos de teste, dando feedback em segundos.

POR QUE BEECROWD?



Gratuito e em português

Mantido por uma comunidade brasileira; interface disponível em PT-BR.



Feedback imediato

Submissão é testada em segundos; você sabe na hora se acertou ou errou.



Categorias por nível

Problemas de iniciante a avançado, permitindo progressão gradual.



Ranking individual

Cada aluno tem seu perfil com estatísticas e ranking global/universitário.

Para nossa disciplina, o BeeCrowd **substitui o laboratório**: vocês resolvem os problemas em casa, no horário que preferirem. O professor acompanha o progresso de cada aluno via ranking da turma, podendo oferecer ajuda pontual a quem travar em algum conceito.



BeeCrowd — antigo URI Online Judge — maior juiz online brasileiro.

Como Usar o BeeCrowd

Agora o professor vai acessar o BeeCrowd ao vivo e demonstrar o ciclo completo: criação de conta, busca de problema, leitura do enunciado, escrita da solução, submissão e leitura do feedback. Acompanhem no projetor. Em casa, vocês farão exatamente o mesmo.

01



Crie sua conta

Acesse beecrowd.com.br e clique em "Register". Use email institucional (recomendado) ou pessoal. Preencha nome, país (Brasil) e universidade. Guarde usuário e senha — vocês usarão o semestre inteiro.

02



Encontre o problema

No menu "Problems", filtre por "Iniciante". Para o Módulo 1, começaremos pelo problema 1000 — "Hello World!". Clique no título para abrir o enunciado completo.

03



Leia o enunciado com atenção

Todo problema tem: **descrição**, **entrada**, **saída**, **exemplos**. Entenda EXATAMENTE o que é pedido antes de codar. Muitos erros iniciantes vêm de não ler o enunciado completo.

04



Escreva a solução no VS Code

Escreva o código no VS Code (**NÃO** diretamente no browser — você perde o autocompletar e não salva localmente). Teste com os exemplos do enunciado antes de submeter. Corrija até passar.

05



Submeta e leia o feedback

Copie o código do VS Code, cole na caixa do BeeCrowd, escolha a linguagem (Python 3) e clique em "Submit". Em segundos, o sistema mostra: **Accepted** (verde), **Wrong Answer** (vermelho) ou outro status.



DEMONSTRAÇÃO AO VIVO: o professor está fazendo o problema 1000 (Hello World!) agora. Notem como ele escreve no VS Code, testa, copia para o BeeCrowd e submete.



Status mais comuns:

Accepted

Wrong Answer

Time Limit

Runtime Error

Compilation Error

Problemas BeeCrowd Recomendados

Para fechar o Módulo 1, cada aluno deve resolver os **5 problemas** abaixo no BeeCrowd até a próxima aula. São todos classificados como "**Iniciante**" — exigem apenas os conceitos vistos nesta aula (print, variáveis, input, condicionais simples). A entrega é **individual** e o prazo é a próxima segunda-feira.

Nº	ID BEECROWD	TÍTULO	CONCEITO	DIFICULDADE
01	1000	Hello World!	print()	Muito fácil
02	1001	Extremamente Básico	variáveis + leitura	Fácil
03	1002	Área do Círculo	variáveis + constantes	Fácil
04	1003	Soma Simples	variáveis + soma	Fácil
05	1005	Média 1	variáveis + média ponderada	Fácil

🔗 PASSOS PARA RESOLVER CADA PROBLEMA

- 1 Acesse beecrowd.com.br → Problems → Iniciante
- 2 Clique no número do problema (ex.: 1000)
- 3 Leia o enunciado, escreva no VS Code, teste localmente
- 4 Submeta no BeeCrowd e valide o **Accepted** verde

💡 DICAS PARA APROVEITAR MELHOR

Resolva os problemas em **ordem crescente de ID** — cada um introduz uma complexidade nova. Leia o enunciado com atenção aos detalhes de formatação. É normal errar nas primeiras tentativas — a persistência é parte do aprendizado.

Resumo e Próximos Passos

Nesta primeira aula estabelecemos as bases conceituais e operacionais para todo o semestre. A aula foi **expositiva com demonstração ao vivo pelo professor** — ambiente, exemplos de código e BeeCrowd. A partir da próxima aula, aprofundaremos cada tópico com novas demonstrações e o BeeCrowd como plataforma de prática contínua.



HOJE

O que cobrimos hoje

- ✓ **Apresentação da ementa completa da disciplina**
- ✓ **Python:** história, filosofia e características técnicas
- ✓ **Vantagens:** comparativo com C e Java, ecossistema
- ✓ **VS Code:** recursos e extensões (demonstração ao vivo)
- ✓ **Instalação:** Python + VS Code (demonstração ao vivo)
- ✓ **Quatro programas clássicos** em Python (demonstração ao vivo)
- ✓ **BeeCrowd:** plataforma oficial de prática apresentada
- ✓ **5 problemas** do Módulo 1 listados como lição de casa



ATÉ A PRÓXIMA AULA

Lição de casa (individual)

- 01 Instalar Python + VS Code**
Reproduzir em casa o passo a passo demonstrado em aula.
- 02 Criar conta no BeeCrowd**
`beecrowd.com.br` — usar email institucional.
- 03 Resolver 5 problemas (M1)**
IDs: `1000`, `1001`, `1002`, `1003`, `1005`
- 04 Criar conta no GitHub**
Publicar o programa `ola.py` no repositório pessoal.
- 05 Explorar python.org**
Tutorial oficial em `docs.python.org/pt-br`.

→ **Próxima aula:** Módulo 2 — Variáveis, Tipos e Operadores. Cheguem com o ambiente instalado e pelo menos o problema `1000` (Hello World!) resolvido no BeeCrowd.



Obrigado!

Bem-vindos ao mundo Python — e ao BeeCrowd.

Esta foi apenas a primeira aula de uma jornada que durará o semestre inteiro. O caminho do programador se constrói com prática, curiosidade e persistência — vocês já deram o primeiro passo. Nos vemos na próxima aula, com ambiente instalado, conta criada no BeeCrowd e muita vontade de aprender.



BEECROWD beecrowd.com.br

"A melhor maneira de aprender a programar é programando." — adaptado de Donald Knuth